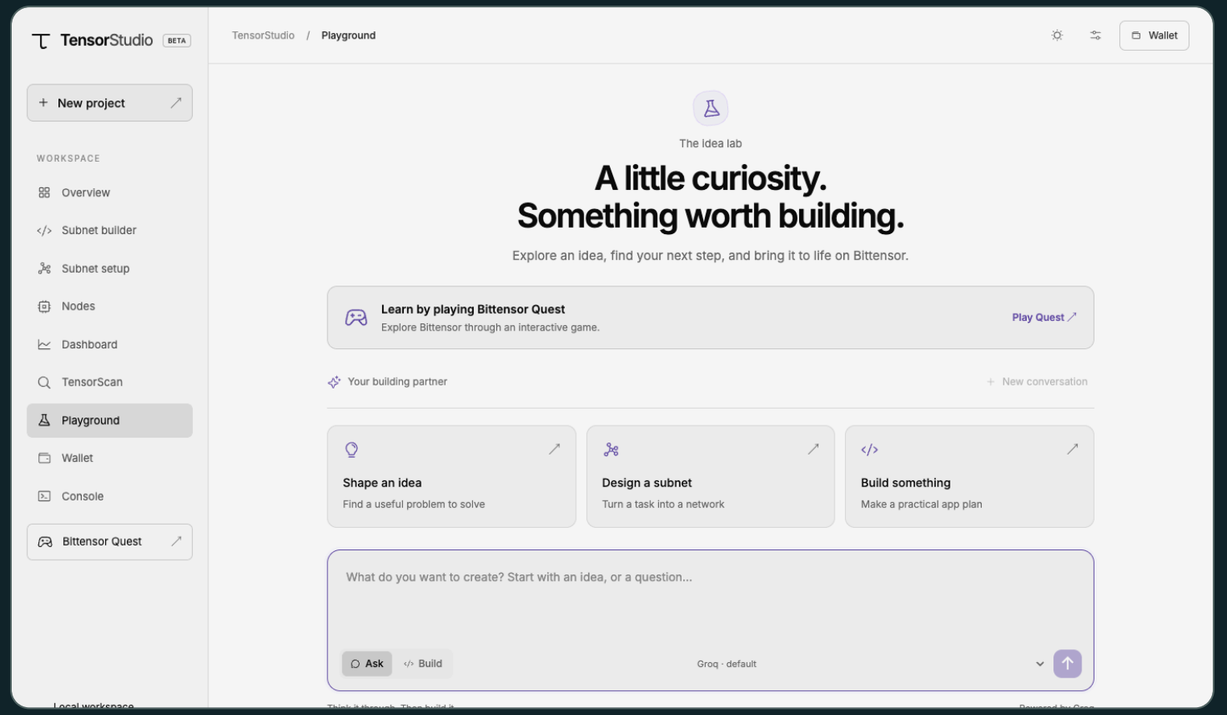
T TensorStudio

One idea. Every next step.

The browser workspace for turning a Bittensor idea into an inspectable experiment.

Learn · Build · Evaluate · Observe

Built by Somesh Talligeri



ACTUAL APPLICATION · GUIDED PLAYGROUND

Make the task clear. Make the evaluation visible. Make the result inspectable.

01 / THE PROBLEM

A useful subnet starts before the first miner runs.

Builders must connect a real problem to measurable work — while navigating tools that live in different places.

01

Understand the network

New builders meet subnets, hotkeys, validators and incentives before they have a clear first experiment.

02

Define useful work

Generated code does not establish quality. The task needs examples, scoring rules and explicit failure cases.

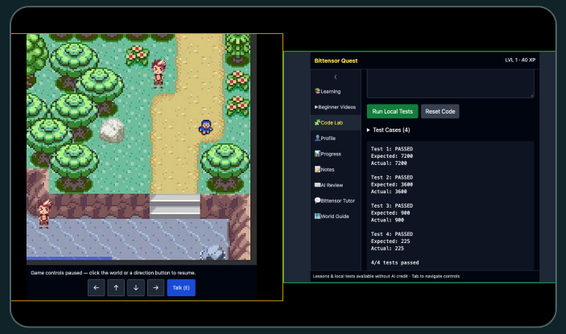
03

Find what really ran

Editors, terminals and explorers tell different stories. A running process alone does not prove network participation.

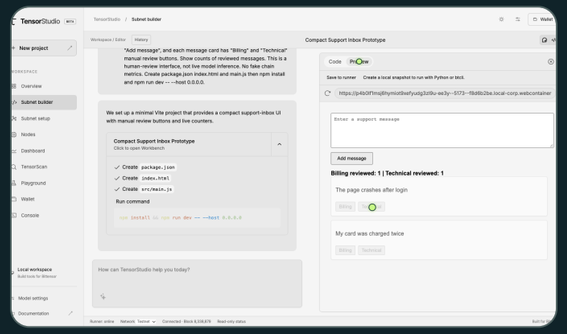
The opportunity Bring learning, implementation, evaluation and observation into a coherent developer journey.

From “what is Bittensor?” to “show me the result.”



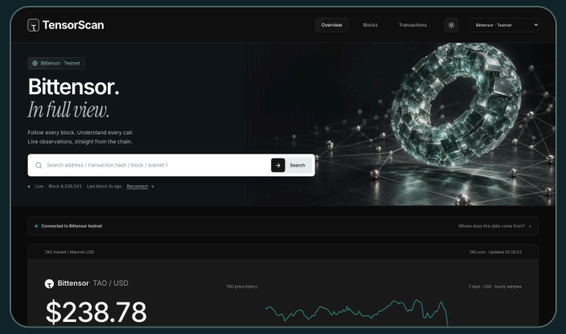
01 · Learn with Quest

Explore a pixel world, answer quizzes and practise concepts in local code exercises.



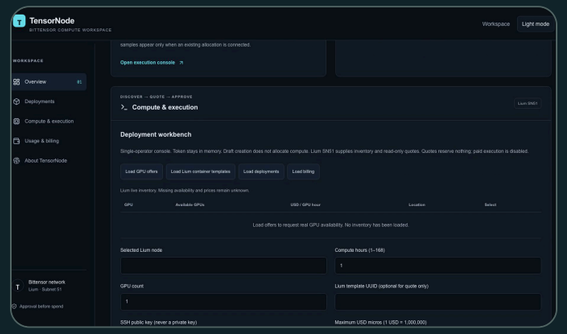
02 · Build in Studio

Clarify the task, generate editable code and interact with a browser preview.



03 · Observe with TensorScan

Inspect real blocks, decoded calls, events, accounts and subnet participants.



04 · Plan operations

Explore compute drafts, recorded events and billing in the separate TensorNode workspace.

Task → Labelled examples → Recipe → Offline scoring → Miner logs → Signed request → Chain observations

Quest and TensorNode are separate companion apps. Studio connects the learning journey; no shared backend integration is implied.

Build around useful work. Keep the evidence connected.

Browser workbench

React / Remix · editable code · browser previews · wallet tools

Web API · Vercel

Guided AI + structured task recipes
Configured inference endpoint · server-only credentials

Subtensor RPC

Independent read-only network observations
Blocks · events · accounts · participants

Authenticated Python runner · Render

FastAPI · project assembly · offline harness · native processes · logs · console

Generated miner → signed development request → response

Bittensor SDK · hotkey identity · inspectable request and result

Why Bittensor?

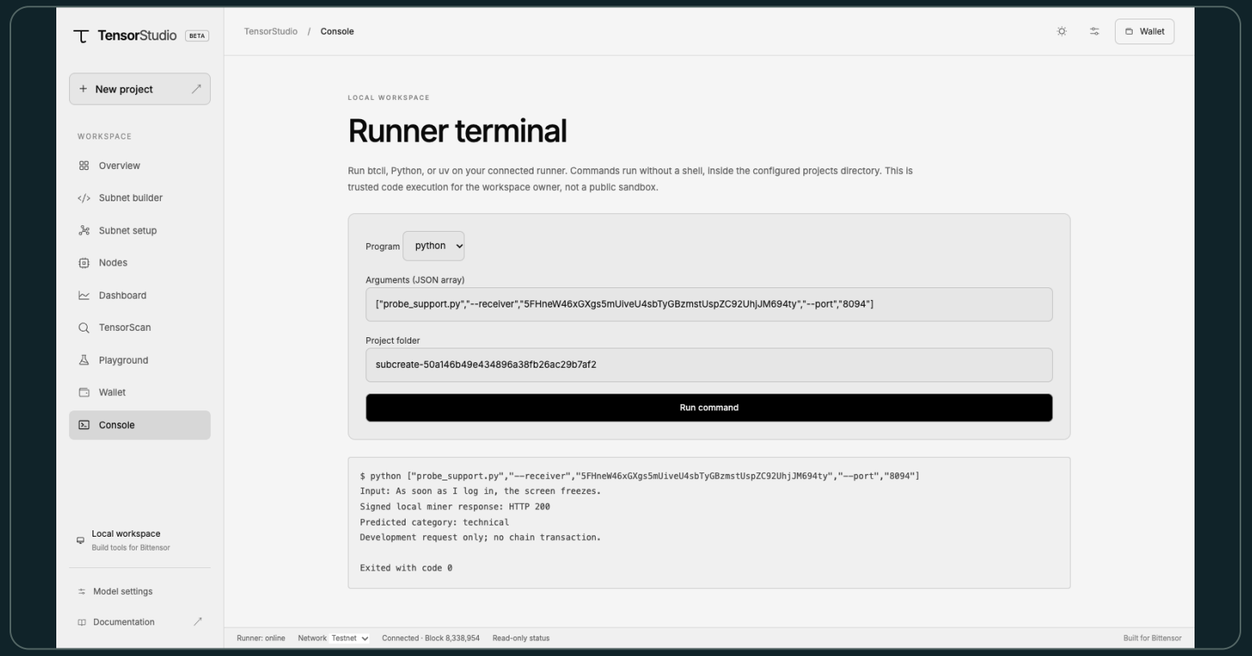
A subnet defines a task. Miners produce work. Validators evaluate it. The network coordinates incentives.

Studio helps builders make that task and evaluation concrete, then inspect their development experiment alongside real chain data.

Bittensor network concepts ↗

Public exploration and bounded AI are available without sign-in. Process execution remains owner-protected. Development inference is distinct from registered subnet participation.

A support-routing idea. An actual response.



ACTUAL CAPTURE · SIGNED MINER REQUEST + RESPONSE

"As soon as I log in, the screen freezes."

MODEL CLASSIFICATION

technical

HTTP 200 · signed development request

- Editable support-inbox prototype with manual review.
- Labelled examples and reviewed project assembly.
- Offline fixture scoring, process logs and console output.
- Independent public testnet observations in TensorScan.

Offline scores use fixture responses and simulated timing. The miner response is real inference. No on-chain registration, reward earnings or paid GPU execution is claimed.

Lower the distance between curiosity and contribution.

For builders at every first step

Newcomers learn the concepts. Developers inspect code and outputs. Subnet teams explore evaluation before committing to network deployment.

A practical path to a business

Proposed paid tiers: team workspaces, isolated managed runners, private evaluation and operational observability.

Business model hypotheses — no customer or revenue claims.

START WITH ONE REAL TASK

Try the workspace. Follow the evidence.

Open TensorStudio ↗

Watch the five-minute developer journey ↗

Studio source · reviewer access required ↗

Bittensor Quest ↗ TensorScan ↗ TensorNode source ↗ Subnet Factory · separate prototype ↗